



구글 LLM

# 기계는 어떻게 언어를 배우는가

인간의 기억에서 LLM까지, 그리고 토큰화·단어 임베딩

---

Part 1. NLP의 역사 · Part 2. 토큰화와 단어 임베딩

14기 이하린

# CONTENTS



Part 1

## NLP의 역사

- NLP의 출발점
- 규칙 기반(ELIZA) → 통계(N-gram)
- RNN → LSTM
- LLM의 등장



Part 2

## 토큰화와 단어 임베딩

- 텍스트를 토큰으로 쪼개는 법
- 원-핫 인코딩과 그 한계
- Word2Vec, GloVe로 의미 학습
- 정적 임베딩의 한계까지



# 우리는 어떻게 기계에게 우리의 말을 이해하도록 가르칠 수 있을까요?

---

이 질문이 바로 **자연어 처리(Natural Language Processing, NLP)** 라는 학문의 출발점입니다.  
NLP는 언어를 계산 가능한 형태로 바꾸고 문맥을 잘 다루도록 발전해 왔습니다.

# NLP(Natural Language Processing)란 무엇인가?

컴퓨터가 인간의 언어를 이해 · 생성 · 조작하도록 만드는 AI 분야입니다.



## 구문 분석 (Syntax Analysis)

문장의 문법 구조 이해  
(품사 태깅, 문장 구조 분석 등)



## 의미 분석 (Semantic Analysis)

단어의 중의성 해소,  
개체 간 관계 파악



## 화용 분석 (Pragmatic Analysis)

맥락 속 실제 의도  
이해하기

→ 챗봇 · 검색엔진 · 스팸 필터링 · 감성 분석 · 기계 번역의 기반이 됩니다.

## 규칙 기반 시대(1950-1970) : ELIZA

언어학자와 프로그래머가 직접 손으로 문법 규칙을 코딩 → 컴퓨터가 그 규칙을 따르는 방식

### 대화 예시

사용자: “나는 슬픔을 느껴요.”

ELIZA: “얼마나 오랫동안 슬픔을 느끼셨나요?”

→ 패턴을 인식해 문장 구조를 재배열할 뿐, 진짜 의미를 이해한 것은 아닙니다.



### 한계

- 언어를 진정으로 이해하지 못함
- 사전에 코딩되지 않은 속어 · 오타 · 새 문법 구조를 만나면 쉽게 실패
- 매우 취약한(brittle) 시스템

# 통계적 전환(1980-2000): N-gram 모델

규칙 대신, 방대한 텍스트(코퍼스)에서 확률적 패턴을 학습

유니그램 (1-gram)

"the", "cat"

바이그램 (2-gram)

"the cat"

트라이그램 (3-gram)

"the cat sat"

핵심: 이전 N-1개 단어를 보고 다음 단어를 확률로 예측 (오늘날 자동완성의 기초 원리)

## 약점: 짧은 문맥만 기억

예문: “그 학생은 매일 아침 학교에 \_\_\_\_”

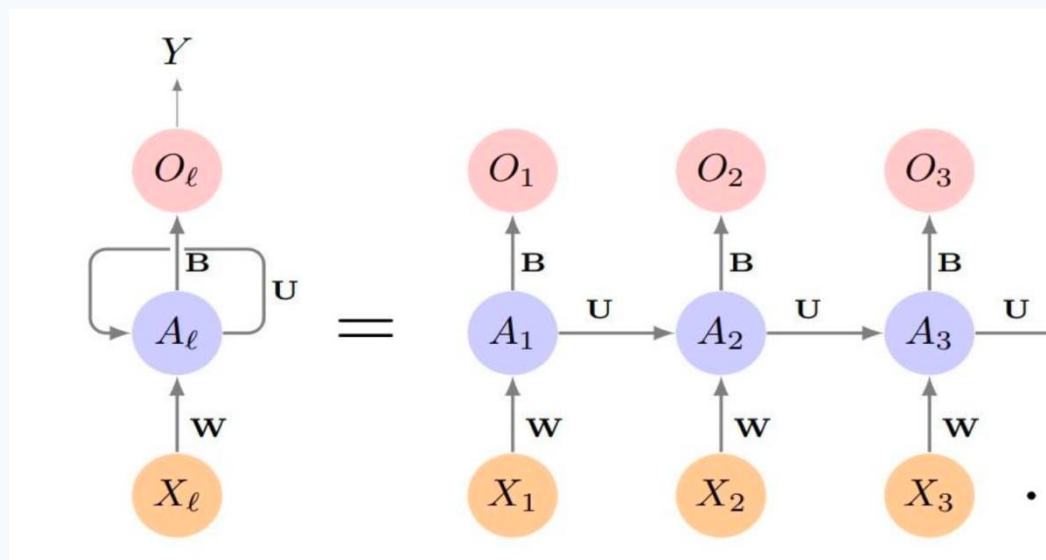
바이그램은 바로 앞 단어 '학교에'만 보고 예측 → '학생은'이라는 주어 정보는 이미 잊혀짐  
→ '갑니다' / '있습니다' / '다닙니다' 중 정답을 보장하지 못함

## 데이터 희소성 문제

N이 커질수록 조합이 폭발적으로 증가 → 대부분 조합이 학습 데이터에 없어 확률이 0이 됨

# 순환 신경망 (RNN)

정보가 지속되도록 하는 '루프(순환)' 구조를 가진 신경망



은닉 상태(기억)가 다음 단계로 계속 전달됩니다→

## 작동 3단계

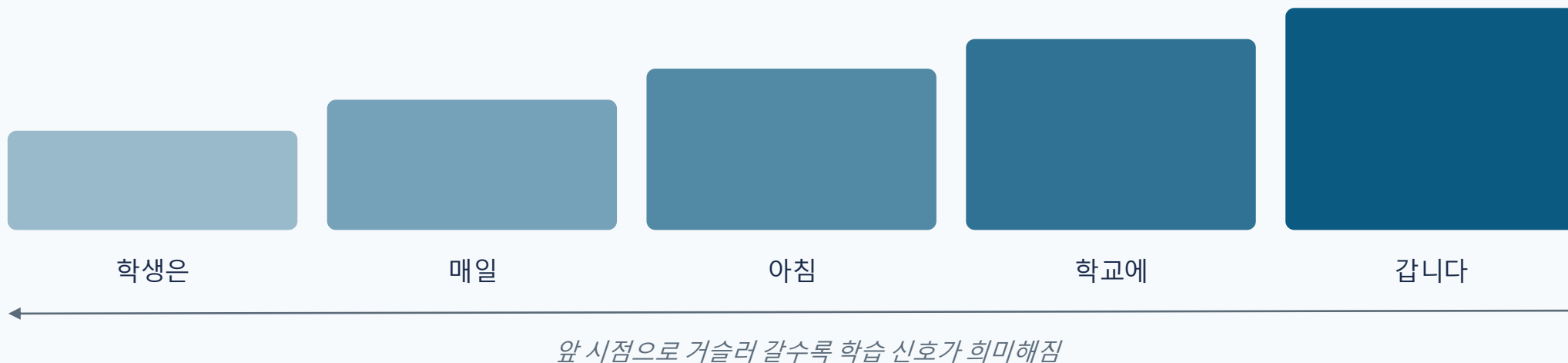
1. 현재 단어(입력)를 받음
2. 이전 은닉 상태(기억)와 합침
3. 새로운 출력 + 새 은닉 상태 생성

**비유** 소설을 읽을 때 이전 장 내용을 기억한 채로 지금 문장을 이해하는 것과 같습니다.

## 기울기 소실 문제

오차(기울기)는 문장 끝에서 처음 방향으로 거꾸로 전파됩니다 (BPTT, Backpropagation Through Time).

학습 신호(gradient)의 세기 →

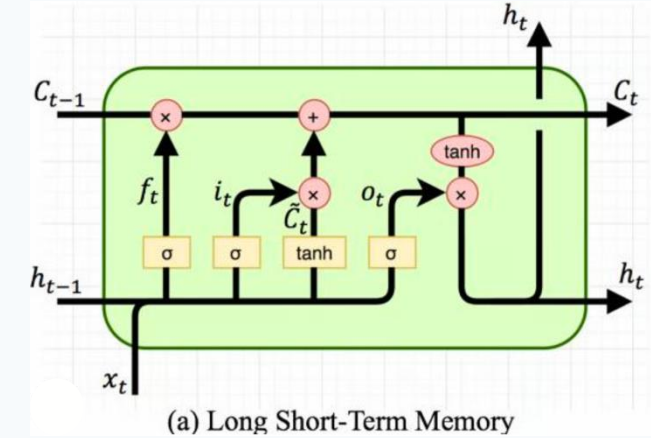


**비유** 뒤쪽에서 전달되는 오차 신호가 앞쪽으로 갈수록 점점 희미해져 메아리처럼 사라짐 → 긴 문장의 앞·뒤 연관성을 학습하기 어려움



## 정보의 문지기: LSTM 게이트 구조

셀 상태 (Cell State) — 컨베이어 벨트처럼 정보가 거의 변형 없이 멀리 전달됨 = 장기 기억



Cell State  $C_{t-1}$   $\longrightarrow$   $C_t$



**Forget Gate**  
망각 게이트 ( $f_t$ )

이전 정보 중 뭘 버릴지 결정  
(0=완전 잊음, 1=그대로 유지)



**Input Gate**  
입력 게이트 ( $i_t$ )

새로 들어온 정보( $x_t, h_{t-1}$ ) 중  
뭘 저장할지 결정



**Output Gate**  
출력 게이트 ( $o_t$ )

지금 뭘 꺼내 쓸지 결정

**비유** LSTM은 '똑똑한 비서'와 같습니다. 모든 걸 받아 적지 않고, 중요한 건 남기고(Input), 불필요한 건 지우고(Forget), 필요한 순간에 꺼내 보고합니다(Output).

## 모델 발전 흐름 한눈에 보기

| 모델                    | 핵심 원리                           | 강점                            | 한계                        |
|-----------------------|---------------------------------|-------------------------------|---------------------------|
| 규칙 기반<br>(e.g. ELIZA) | 수작업으로 만든<br>언어 규칙과 패턴 매칭        | 알려진 패턴에 대해<br>해석 가능 · 정밀함     | 새 표현에 취약,<br>진정한 이해 없음    |
| N-gram<br>(통계)        | 이전 N-1개 단어에 기반한<br>다음 단어의 확률 예측 | 단순 · 효율적,<br>노이즈가 있는 데이터에 강건함 | 짧은 문맥만<br>포착              |
| RNN<br>(순환 신경망)       | 과거 순서열의 기억 역할을 하는<br>은닉 상태 루프   | 이론상 긴<br>의존성 가능               | 실제로는 기울기 소실로<br>단기 기억에 그침 |
| LSTM<br>(장단기 기억)      | 게이트 + 셀 상태<br>(망각, 입력, 출력)      | 장기 의존성<br>학습 가능               | RNN보다 복잡,<br>계산 비용 ↑      |

# LLM의 등장

LSTM은 긴 문맥을 완전히 이해하기 어렵고, 병렬 처리가 안 돼 대규모 학습에 제약이 있었습니다.  
이를 극복한 것이 **Transformer**와 이를 바탕으로 한 **거대언어모델(LLM)** 입니다.

## 수조

개 단어 규모의  
학습 데이터

## 수십억

개의  
파라미터

## 병렬

트랜스포머의 셀프 어텐션으로  
전체 시퀀스 동시 처리

### 한 줄 역사 정리





Part 2

# 토큰화와 단어 임베딩

텍스트에서 의미로

LLM의 근본 원리는 '다음 단어 예측'. 그런데 컴퓨터는 단어 자체를 이해하지 못합니다.

→ 단어를 숫자로 바꿔야 학습과 예측이 가능합니다.

# 토큰화(Tokenization)

"I love playing football." →  
 ["I", "love", "play", "##ing", "foot", "##ball", "."] →  
 [146, 512, 237, 207, 334, 238, 101]

| 특징      | 단어 토큰화            | 문자 토큰화            | 서브워드 토큰화              |
|---------|-------------------|-------------------|-----------------------|
| 핵심 아이디어 | 공백 기준 분할          | 개별 문자로 분할         | 하위 단위로 분할             |
| 어휘집 크기  | 매우 큼              | 매우 작음             | 중간                    |
| oov 처리  | 취약함<br>(<UNK> 사용) | 완벽함<br>(OOV 없음)   | 뛰어남<br>(새 단어 분해)      |
| 시퀀스 길이  | 가장 짧음             | 가장 길              | 중간                    |
| 의미 단위   | 높음<br>(단어 수준 의미)  | 낮음<br>(내재적 의미 없음) | 중간<br>(subword 수준 의미) |

## 단어 토큰화

"Never give up." →  
 ["Never", "give", "up", "."]

**장점** 간단 · 직관적

**약점** 어휘 폭증, 미등록 단어(oov) 취약

## 문자 토큰화

"smarter" →  
 [s, m, a, r, t, e, r]

**장점** oov 문제 해결, 오타에 대한 강건성

**약점** 의미 손실, 긴 시퀀스

## 서브워드 토큰화

"tokenization" →  
 [token, iz, ation]

**장점** 현대 표준 (BERT, GPT)

**약점** 구현이 더 복잡

# 서브워드 토큰나이저

## BPE

Byte-Pair Encoding

가장 자주 등장하는 인접 쌍을 반복 병합

## WordPiece

우도 기반 병합

병합 시 훈련 데이터의 우도(likelihood)를 최대화하는 쌍 선택

$$\text{Score}(x, y) = \frac{P(x, y)}{P(x) P(y)}$$

$P(x, y)$ : 두 토큰  $x, y$ 가 나란히 등장할 확률  
 $P(x)$ : 첫 번째 토큰이 등장할 확률  
 $P(y)$ : 두 번째 토큰이 등장할 확률  
 비율이 높으면 강하게 연결된 쌍

## SentencePiece

언어 독립적 프레임워크

공백까지 문자처럼 취급 → 무손실 · 언어 독립적

### BPE 병합 과정 예시

말뭉치: 'low\_'x5, 'lower\_'x2, 'newest\_'x6, 'widest\_'x3

1. 초기화 → 모든 단어를 문자 단위로 분해 { l, o, w, e, r, n, s, t, i, d, \_ }

2. 가장 빈번한 인접 쌍을 찾아 병합 → 어휘집에 추가  
 (e, s) → "es" (9회 등장) , (es, t) → "est" (9회 등장)

3. 목표 어휘집 크기에 도달할 때까지 반복

# 원-핫 인코딩

## 원-핫 인코딩

크기가  $v$ 인 어휘집을 만든 후 각 토큰을  $v$  차원 벡터로 표현

예시) 어휘집 { cat:0, dog:1, car:2 }

cat [1, 0, 0]

dog [0, 1, 0]

car [0, 0, 1]



### 차원의 저주

벡터 길이 = 어휘집 크기  
→ 메모리 폭발



### 직교성의 저주

모든 벡터가 서로 직각  
→ 의미 관계 표현 불가



### 희소성의 저주

대부분이 0으로 채워짐  
→ 정보 밀도 없음

→ 이 한계를 극복하려고 등장한 것이 **단어 임베딩(Word Embedding)** 입니다.

정리: **문장** → **토큰화** → **Token ID(단순 번호표)** → **Embedding** → **의미를 가진 벡터**

※ 원-핫 인코딩은 Token ID를 벡터로 표현하는 초기 방식이며, 현대 모델에서는 ID로 임베딩 벡터를 바로 조회하는 방식이 일반적

## 분포 가설과 Word2Vec

**분포 가설** "단어의 의미는 함께 등장하는 주변 단어를 통해 알 수 있다."

예시) "커피"와 "차"는 둘 다 "나는 \_\_ 한 잔을 마시고 싶다"에 자주 등장  
→ 의미가 비슷하다고 추론

### CBOW (Continuous Bag-of-Words) : 주변 → 중심 예측

"The cat \_\_\_\_ on the mat"  
→ "sat" 예측

**장점** 훈련 속도 빠름

**단점** 희귀 단어 정보 희석

### Word2Vec

지역적 문맥 창을 기반으로 주변 단어를 예측하도록 얇은 신경망을 학습시켜, 비슷한 문맥에 등장하는 단어들이 가까운 벡터로 표현되도록 임베딩을 학습하는 방법

### Word2Vec의 임베딩 학습 원리

1. 단어를 원-핫 벡터로 변환
2. 임베딩 벡터 조회
3. 중심 단어 ↔ 주변 단어 예측
4. 예측 오차를 이용해 가중치 업데이트
5. 의미를 반영하는 임베딩 형성  
→ 행렬에는 토큰마다 하나의 벡터가 있고, 예측을 잘하도록 그 숫자들이 계속 수정됨

### Skip-gram : 중심 → 주변 예측

"sat" → the, cat, on, mat  
예측

**장점** 희귀 단어 처리에 강함

**단점** 훈련 속도 느림



# 전역적 통계로부터의 학습: GloVe

핵심 아이디어: 확률 그 자체보다 "**비율**"이 단어 간 의미 차이를 더 잘 드러냅니다.

| 단어         | ice와의 관계            | steam과의 관계            |
|------------|---------------------|-----------------------|
| solid (고체) | ice와 잘 어울림 → 확률 큼   | steam과는 안 어울림 → 확률 작음 |
| gas (기체)   | steam과 잘 어울림 → 확률 큼 | ice와는 안 어울림 → 확률 작음   |
| water (물)  | ice와도 자주 등장         | steam과도 자주 등장         |

**Word2Vec** = 지역(local) 관점

짧은 구문 패턴에 강함

**GloVe** = 전역(global) 관점

king-queen 같은 큰 의미 관계에 강함

|       | Word2Vec (Skip-gram) | GloVe             |
|-------|----------------------|-------------------|
| 모델 계열 | 예측 기반, 지역적 문맥 창      | 카운트 기반, 전역 행렬 분해  |
| 장점    | 미세한 구문 패턴, 희귀 단어 우수  | 광범위한 의미 관계, 빠른 훈련 |
| 단점    | 전역 통계 미활용            | 초기 행렬 구축에 메모리 多   |

# 임베딩의 놀라운 속성, 그리고 한계

## 유추 관계 (Semantic Arithmetic)

$$\text{vector}(\text{king}) - \text{vector}(\text{man}) + \text{vector}(\text{woman}) \approx \text{vector}(\text{queen})$$

→ '왕족'이라는 추상 개념이 벡터 축으로 조직화되어 있음을 보여줌 (분포 가설의 강력한 증거)



### 정적 임베딩의 다의어 문제

"배가 아파요" → 배 = 신체 부위

"배를 먹었어요" → 배 = 과일

정적 임베딩은 이 둘을 완전히 같은 벡터로 취급!



### 필요한 것: 문맥적 임베딩

단어 의미는 문맥에 따라 달라져야 함

→ 이를 해결한 것이 Transformer,  
그 핵심 아이디어가 Attention

다음 시간: Transformer와 Attention 메커니즘으로 이어집니다 →

**감사합니다**